SENTINEL Lab 4 Brief: `libsync`

ECE 476 Engineering System Software • Fall 2026 • University of Victoria

Lab session	Friday, December 4, 2026
Prelab due	Wednesday, December 2, 2026, by midnight, on Brightspace. Required, with no marks[1]
Deliverable due	Friday, December 11, 2026, on Brightspace: 4% of the grade, out of 80 marks
Video due	Friday, December 11, 2026, on Brightspace, with the deliverable: 4% of the grade, out of 20 marks (Section 6.2). Lab 4 has no oral exam
Lab 4 total	8% of the course grade: the deliverable and the video

This brief is the one paper of Lab 4. It states what no other paper and no repository file states, and a footnote names the place of each other fact. Read it with `src/libsync/include/sentinel/sync.h`[2], which states each function: its steps in order and its errors.

1 The task

Lab 3 showed the problem: with no lock, eight workers on two processors or more lost updates, took one report two times, and drove the count of the queue below 0, so the stop never ended. Lab 4 builds `libsync`, which solves it with the three tools of the lectures: a mutex, two semaphores and a lock order. It does the work of R4, shared data that stays exact,[3] and R5, a centre that never freezes.[4]

Write three files in `src/libsync/src/`, and one test (Section 5). Each file comes with a stub and the steps of each function that you write:

- `safetrack.c`: `safe_track_init`, `safe_track_destroy` and `safe_track_update`. Lecture 4.6, Mutex.
- `sem_queue.c`: `sem_queue_init`, `sem_queue_destroy`, `sem_queue_push` and `sem_queue_pop`. Lecture 3.3, Semaphores. The file comes with `wait_for_slot()`, `is_shut_down()` and `pop_after_shutdown()` written: do not change them.
- `resources.c`: `resources_acquire_both` and `resources_release_both`. Lecture 4.8, Deadlocks.

The course gives the other files as source. Read them, and do not change them: `src/libsync/Makefile`, `sync.h`, the files in `src/libsync/given/` (the worker pool, the rest of the track picture, of the queue and of the resources, and the picture divided into cells), and `src/sentinel/`, the final program.[5] Lab 4 uses your libraries of Labs 1, 2 and 3: your `sem_queue.c` calls your `queue_push` and `queue_pop` of Lab 3. A fault there is a fault in Lab 4.

2 Lab 4 in SENTINEL

Lab 4 completes SENTINEL: the four libraries make one centre, `build/sentinel`, which reads the four sensors, puts each report in the queue, and lets 8 workers update the track picture. Each part of your code removes one fault of Lab 3:

[1] The Lab 4 prelab, `docs/lab4/prelab4.pdf` in your repository. It is due before the Lab 4 session.

[2] `src/libsync/include/sentinel/sync.h`, in your repository. It states what each function of Lab 4 does, in order, with each of its errors.

[3] R4 is a requirement of SENTINEL: many workers write the same shared record, and must not corrupt it. The SENTINEL Lab Manual, Section 1.2, `docs/manual/sentinel_manual.pdf`, lists the six requirements.

[4] R5 is a requirement of SENTINEL: the centre must never freeze while it still looks alive. The SENTINEL Lab Manual, Section 1.2, `docs/manual/sentinel_manual.pdf`, lists the six requirements.

[5] `build/sentinel` is the final program: the four libraries in one process, from the given `src/sentinel/main.c`. At its end it prints one line for each requirement, R1 to R6, with PASS or FAIL.

The fault of Lab 3	Your code of Lab 4	What you see
Lost or repeated updates of the track picture	`safe_track_update`: one mutex around `track_update`	`R4 shared data stays exact PASS`: the reports processed are the reports counted plus the reports refused
`count` of the queue below 0, and a stop that never ends	`sem_queue_push` and `_pop`: two semaphores and a mutex around your Lab 3 queue	The ledger says `BALANCED`, and the run stops
A worker that polls an empty queue with `usleep`	`sem_wait` on `full`: a worker sleeps until a report comes	No processor time is used for an empty queue
Two workers that take two locks in opposite orders	`resources_acquire_both`: one fixed order	`R5 never freezes PASS`

The run of 8 workers with all processors (`--cpus all`) that disagreed in Lab 3 agrees in Lab 4, in each run.

3 The ideas

3.1 A critical section, and a mutex

A *critical section* is code that reads and writes shared data, and that must run as one step for each other thread. `track_update` is one: it finds an entry, changes it, and adds to the totals. A *mutex* lets one thread at a time into the critical section: `pthread_mutex_lock()` waits until no other thread holds it, and `pthread_mutex_unlock()` lets the next thread in. Lock immediately before the critical section, and unlock immediately after it: a lock that covers less lets the race back in, and a lock that covers more makes the other threads wait for no reason. Release the lock on each path, also after an error: a thread that returns with the lock held stops each other thread.

3.2 A semaphore, not a sleep

A *counting semaphore* holds a number. `sem_wait()` takes 1 from it, and when the number is 0 the thread sleeps, with no use of the processor, until another thread calls `sem_post()`, which adds 1. In Lab 3 a worker with an empty queue called `usleep(200)` and tried again: it used the processor to find nothing. In Lab 4 it sleeps in `sem_wait()`, and it wakes when a report comes.

3.3 The bounded buffer

The queue of Lab 4 is the bounded buffer of Lecture 3.3, around your FIFO ring of Lab 3. Two semaphores count: `empty` the free slots, and `full` the reports that wait. A push waits on `empty` and posts `full`; a pop waits on `full` and posts `empty`. A mutex still covers the call of `queue_push` or `queue_pop`, because two threads that pass the semaphores must not be in the ring at the same time. The order is fixed: `sem_wait` first, then the lock. A thread that takes the lock and then sleeps in `sem_wait` holds the lock while it sleeps, and no other thread can post. A full queue of Lab 4 does not refuse a report as Lab 3 did: the centre waits, and the program counts each wait.

3.4 Four conditions, and one order

A *deadlock* needs four conditions at the same time (Lecture 4.8): mutual exclusion, hold and wait, no preemption, and a circular wait. A worker that takes the tracking sensor and then the HQ data link, and another worker that takes them in the opposite order, can each hold one and wait for ever for the other: a circular wait. The centre then freezes, and each thread is still alive (R5). Remove one condition, and no deadlock can occur. `resources_acquire_both` removes the circular wait: it takes the two locks by rank, the lower rank first, whatever order the caller names. The other three conditions stay: a mutex is mutual exclusion, a worker needs both resources at the same time, and only the holder of a mutex can unlock it.

4 The code rules

The code rules of the Lab 1 Brief[6] apply to Lab 4 too. `sync.h`[7] gives the steps of each function, and each starter file repeats them above the function. The marks of the code rules (Section 7) check these steps. Four rules are new:

- Each `pthread_mutex_lock` has its `pthread_mutex_unlock` on each path.
- `sem_wait` comes before the lock, and `sem_post` after the unlock.
- Check the arguments before you take a lock: an error must not leave a lock held.
- Two locks are taken in the order of their ranks, the lower first.

A `pthread_*` function returns its error number, and does not set `errno`: set `errno` to that number, as the steps say. When an `init` fails after it made a part, it destroys that part first. `sync.h` lists the CALLS THAT NEED NO CHECK, such as `pthread_mutex_lock`, with the reason of each. One rule of Lab 1 changes: a call that a step marks *No check* needs no branch and no comment. The step gives the reason.

5 Build, run and check

Lab 4 adds these targets to those of the SENTINEL Lab Manual.[8] Run each one at the top of your repository.

Command	What it does
`make`	Builds `src/libsync` with the given `Makefile`, and then `build/sentinel`. Your three files come as stubs, so they build at once, and each stub fails until you write it
`make lab4`	Runs `build/sentinel` with your libraries for 3 s, for the page[9]
`make accept4`	Runs the 16 checks of `tools/accept/accept_lab4.c` on your library and the final program
`make submit LAB=4`	Writes the zip[10] of Lab 4 (Section 6)

These are the last rows of `make accept4` when each check passes:

```
  safetrack.c      6 of 6   100%
  sem_queue.c      6 of 6   100%
  resources.c      3 of 3   100%
  sentinel         1 of 1   100%

16 of 16 checks pass (100%).
```

A failed check prints a sentence about the contract. Read the check in `accept_lab4.c` to see what it does. Most checks need no luck: to see that your function takes a lock, the test holds that lock first, and a second thread calls your function, which must wait. A check with a call that does not return ends after 30 s with *no result*: a deadlock, or a lock that was never released. Do not change the file: the marks come from the course copy of it.

Your test. `src/libsync/tests/test_sync.c` comes with its first case written. A comment above each other test function states what to check: two threads that update one contact 100000 times each, the order of the queue, and `NULL`. Write their bodies with `assert()`, as the Lab 1 Brief[11] describes. The marker reads each case, and runs the test. `make test` builds and runs it. With no lock, the case of two threads fails on two processors or more: that is the fault of Lab 3, and your test proves that it is gone.

valgrind. This command checks the threads and the memory of the centre:

[6] The rules of the code of Lab 1, which apply to each later lab: the Lab 1 Brief, Section 2, `docs/lab1/lab1_brief.pdf`.

[7] `src/libsync/include/sentinel/sync.h`, in your repository. It states what each function of Lab 4 does, in order, with each of its errors.

[8] The make targets of all labs, such as `make test` and `make watch`: the SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`.

[9] The page is the operations centre display. Start it with `make watch`, and open `http://localhost:8080/`. `web/README.md`, in your repository, describes the page and each view.

[10] `make submit` builds your work from clean, tests it, and writes the zip. The Submission Guide, Section 5, `docs/submit/submit.pdf`, describes it, and Section 6 the upload.

[11] Your own tests are C programs with `assert()` and no framework. The Lab 1 Brief, Section 4.2, `docs/lab1/lab1_brief.pdf`, states their rules.

```
SENTINEL_PROFILE=world valgrind --leak-check=full --child-silent-after-fork=yes \
    --error-exitcode=1 build/sentinel --workers 2 --ms 3000
```

The final program forks a helper process, which ends with `_exit` and keeps a copy of the memory of the pool; `--child-silent-after-fork=yes` reports the centre only. Read its `ERROR SUMMARY` and its leak summary: with an error or a leak, it exits with status 1. `valgrind` runs one thread at a time, so it finds a leak, not a race.

6 Hand it in

The hand-in of Lab 4 is your code, one file of your own run in `docs/lab4/`, and a video. `make submit`[12] with `LAB=4` puts the code and the file in the zip. Upload the zip and the video together.

6.1 The record run

Run the final program with 8 workers and all processors (`--cpus all`), for 20 s, with your own libraries, at the top of your repository. It needs two processors or more: `nproc` gives the number. If it gives 1, give your virtual machine 2 processors or more in its settings.[13] `SENTINEL_PROFILE=world` gives the four sensors the traffic of one world, which each one sees.[14] Without it, each sensor sends positions of its own, and the centre refuses most reports as impossible.

```
make
SENTINEL_PROFILE=world build/sentinel --workers 8 --cpus all --ms 20000 \
    --log build/lab4.log > docs/lab4/sentinel_8workers.txt
```

The log in `build/` is not a part of the hand-in. This is the end of the file from one run of this command. Your numbers differ from run to run. SAT sends in passes,[15] so 20 s give about 20000 reports of SAT:

```
    accounted for                  21384   BALANCED
  R1 four sensors, one may die     PASS (4 of 4 reported, 4 still running at the end)
  R2 priority overtakes routine    PASS
  R3 absorbs SAT at 2000/s         PASS (SAT sent 19882, 1988/s read while it sent)
  R4 shared data stays exact       PASS (21384 processed = 21381 counted + 3 refused, ...
  R5 never freezes                 PASS (longest pause 24 ms, poll timeout 100 ms)
  R6 every action timestamped      PASS (build/lab4.log)
```

The checks of agreement. The marker compares the file and your code: `sentinel_8workers.txt` has the line `workers` with 8, a ledger that says `BALANCED`, and `PASS` on each line from R1 to R6. Compare it with your `docs/lab3/demo_8workers.txt` of Lab 3 (if it is lost, run the command of Section 6.1 of the Lab 3 Brief[16] again): the same 8 workers on the same processors. The last line of Lab 3 says `DISAGREES` or `THE WORKERS DID NOT STOP`; Lab 4 says `BALANCED` and `R4 ... PASS`.

6.2 The video

The video replaces the oral exam: 4% of the course grade, out of 20 marks. Record your screen and your voice, at most 5 minutes, as an MP4 file, and upload it on Brightspace with the zip, by Friday, December 11, 2026. Any screen recorder that writes MP4 is correct, for example OBS Studio (Linux, Windows and macOS); the recorder of macOS (Shift-Command-5) writes a `.mov` file, which you export as MP4. Use this script:

1. **The whole project** (about 2 minutes). Run the command of Section 6.1 with `--ms 10000`, and with no `> docs/lab4/...`, so that the output is on the screen and your hand-in file stays as it is. Show the ledger and the six lines from R1 to R6. Say what SENTINEL does, which lab built each part, and what each requirement means.
2. **The problems that you solved in Lab 4** (about 3 minutes). Show your `safe_track_update`, your `sem_queue_push` and `sem_queue_pop`, and your `resources_acquire_both`. For each one, say what goes wrong

[12] `make submit` builds your work from clean, tests it, and writes the zip. The Submission Guide, Section 5, `docs/submit/submit.pdf`, describes it, and Section 6 the upload.

[13] The Setup Guide, `docs/setup/setup.pdf` in your repository. It sets up your machine and your own copy of the repository.

[14] A profile selects the traffic that `contactfeed` sends: `world`, `ideal`, `dirty`, `field` or `threat`. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes each one.

[15] SAT sends in passes: 2000 reports each second for 10 s, then no report for 10 s. The SENTINEL Lab Manual, Section 1.3, `docs/manual/sentinel_manual.pdf`.

[16] The Lab 3 Brief, `docs/lab3/lab3_brief.pdf` in your repository. It states the contract of Lab 3, its ideas and its hand-in.

without it, and how your code prevents it. For the lock order, say which condition of a deadlock your order removes.

Speak in your own words, and show your own code and your own runs. A video longer than 5 minutes is marked on its first 5 minutes.

7 Done when, and the marks

☐ `make` prints no warning, and `make test`[17] passes your test.
☐ `make accept4` gives 16 of 16.
☐ `docs/lab4/sentinel_8workers.txt` agrees as Section 6.1 states.
☐ `valgrind` reports no error and no leak.
☐ The conditions of a finished Lab 1 in the Lab 1 Brief[18] still hold, and `make accept3` still gives 14 of 14.
☐ The video is at most 5 minutes, and follows the script of Section 6.2.

Part of the deliverable	**Marks**
Acceptance test (`make accept4`), in proportion to the checks that pass	20
Course tests: the tests of the course, on each function of `sync.h`[19] that you write	24
Your test (Section 5)	12
Code rules, and the errors of each call (Section 4)	12
`valgrind`: no error and no leak	4
Agreement of the hand-in file (Section 6.1)	8
Total	**80**

Part of the video	**Marks**
The whole project: the run, the part of each lab, and what R1 to R6 mean	8
The problems that you solved in Lab 4, with your own code	12
Total	**20**

You do not get the course tests. They check only what `sync.h` and this brief state.

[17] The make targets of all labs, such as `make test` and `make watch`: the SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`.
[18] The conditions of a finished Lab 1, such as no warning and a clean `valgrind`: the Lab 1 Brief, Section 5, `docs/lab1/lab1_brief.pdf`.
[19] `src/libsync/include/sentinel/sync.h`, in your repository. It states what each function of Lab 4 does, in order, with each of its errors.